

Atividade 1: Arquitetura de E-Commerce Distribuído

Disciplina:	Sistemas Distribuídos	Data de Entrega:	05 de Junho de 2026
Estudante:	Guilherme Vinícius Rangel Silva	Status do Sistema:	100% Funcional (Com Bônus Completos)

1. Implementação da Comunicação Inter-serviços

A comunicação entre os componentes do ecossistema distribuído foi implementada de forma híbrida utilizando o protocolo **HTTP/REST**. O tráfego externo originado pelo cliente é centralizado unicamente no componente **API Gateway** (porta pública **5000**), que atua como um Proxy Reverso Assíncrono construído sobre a biblioteca **httpx**. O Gateway analisa o prefixo da URL da requisição e a encaminha dinamicamente para o microserviço interno correspondente através da rede isolada do Docker.

A comunicação interna de negócio (lógica entre microserviços) ocorre de forma síncrona. Quando o **Serviço de Pedidos** (porta **5003**) recebe uma intenção de compra, ele assume o papel de cliente interno e dispara requisições HTTP diretas via biblioteca **requests** para validar o ciclo de vida do usuário no **Serviço de Usuários** (porta **5001**) e consultar a precificação do item diretamente no catálogo mantido pelo **Serviço de Produtos** (porta **5002**).

2. Estratégia de Consistência e Replicação de Dados

Para o armazenamento do Serviço de Produtos, foi estritamente adotada a estratégia de **Consistência Forte** (Consistência Cega ou Baseada em Quórum Síncrono de Escrita). O sistema gerencia duas réplicas distintas em containers isolados: a Réplica A (porta **5002**, considerada a instância de entrada primária pelo Gateway) e a Réplica B (porta **5012**).

Sempre que uma operação de escrita (criação de novo produto via método POST) atinge o ecossistema, o microserviço realiza o espelhamento imediato e síncrono do payload em ambas as bases JSON locais. A requisição é bloqueada e mantida em espera até que ambos os arquivos físicos registrem o dado com sucesso. O código HTTP de sucesso (**200 OK**) só é devolvido ao cliente externo após a confirmação integral de escrita de ambas as réplicas. Essa abordagem sacrifica a latência em prol da integridade, garantindo o cumprimento do Teorema CAP para consistência imediata: qualquer leitura subsequente distribuída trará dados idênticos e sem divergências temporais.

3. Tolerância a Falhas e Comportamento do Mecanismo Heartbeat

A tolerância a falhas foi projetada seguindo o princípio da **Degradação Graciosa**, gerenciada por um subprocesso assíncrono e concorrente no API Gateway que executa checagens periódicas (*Heartbeat*) a

cada 5 segundos. Caso o **Serviço de Pedidos** (porta **5003**) sofra uma interrupção crítica ou caia, o ecossistema reage da seguinte forma:

- O Gateway falha em obter resposta no endpoint **GET /health** por duas tentativas consecutivas e imediatamente atualiza o mapa de saúde interno do serviço para o status **"offline"**, registrando o evento no console acompanhado de um timestamp preciso.
- Qualquer requisição subsequente direcionada às rotas de pedidos é interceptada preventivamente pelo Gateway, que barra o processamento e retorna o código de erro padrão **HTTP 503 Service Unavailable**, impedindo estouros de timeout ou falhas ocultas.
- Os microsserviços de **Usuários** e **Produtos** permanecem 100% operacionais de forma isolada devido ao desacoplamento físico das instâncias. O usuário final mantém a capacidade de efetuar cadastro, autenticação e visualização do catálogo, perdendo transitariamente apenas a capacidade de finalizar compras até que o Gateway registre a recuperação automática e o retorno do serviço de pedidos.

4. Segurança Descentralizada e Controle de Acesso via JWT

A proteção das rotas contra acessos não autorizados apoia-se em tokens **JWT (JSON Web Tokens)** emitidos pelo Serviço de Usuários após autenticação bem-sucedida (com senhas previamente criptografadas em hash via algoritmo de segurança robusto **bcrypt**). O token gerado é assinado digitalmente utilizando uma chave secreta e o algoritmo **HS256**.

Estrutura de Segurança: O payload imutável do token transporta explicitamente as alegações de identidade do usuário e seu nível de privilégio, definido pelo campo de controle **"role": "user"** ou **"role": "admin"**.

Quando um usuário comum tenta submeter uma requisição para o endpoint de criação de novos produtos (**POST /products**), o token é obrigatoriamente exigido no cabeçalho. O microsserviço de produtos decodifica o payload e valida a integridade da assinatura criptográfica usando a chave secreta compartilhada. Ao inspecionar o campo **"role"** e constatar o valor **"user"**, o servidor imediatamente interrompe a execução da lógica de negócios e responde com erro de negação de privilégio (**HTTP 401/403**). Como o cliente não possui acesso à chave secreta, qualquer tentativa de adulterar o payload localmente no navegador corrompe a assinatura digital, tornando a validação robusta e imutável.

5. Limitações Críticas Frente a um Cenário de Produção Real

Apesar de atender plenamente os requisitos acadêmicos e arquiteturais, o projeto apresenta limitações técnicas para sustentar cargas de trabalho em produção:

- **Camada de Persistência Inadequada:** O uso de arquivos estáticos em formato JSON gera sérios gargalos de concorrência e I/O sob múltiplos acessos simultâneos. Não há mecanismos nativos de travas (locks) ou isolamento de transações ACID. Um cenário real exigiria migração para SGBDs distribuídos estruturados (PostgreSQL) ou não-estruturados (MongoDB).
- **Latência por Acoplamento Síncrono:** A orquestração síncrona de pedidos via chamadas HTTP diretas gera acoplamento temporal e propaga falhas em cascata se a rede interna apresentar lentidão. Sistemas

de alta escala exigem mensageria assíncrona orientada a eventos utilizando corretores como RabbitMQ ou Apache Kafka.

- **Fragilidade no Gerenciamento TLS/HTTPS:** A infraestrutura utiliza certificados digitais autoassinados gerados em tempo de compilação dentro dos containers. Isso exige desativar a verificação rígida das requisições (`verify=False`) no código Python para viabilizar o tráfego. Em produção, os certificados devem ser gerenciados por Autoridades Certificadoras públicas ou automatizados por uma malha de serviços (*Service Mesh*) em ambiente de orquestração Kubernetes.